

INTRODUCTION TO COMPUTER PROGRAMMING WITH PYTHON

INDIVIDUAL PROJECT- LIBRARY SYSTEM

Module: Python Programming

Student Name: Divakar Sabarwal

Student ID: GH1047352

Submitted To: Mr. RAMIN BAGHAEI MEHR

GIThub Repository:

https://github.com/sabarwaldivakar/Library_Project.git

TABLE OF CONTENTS

SR. NO.	TOPIC	PAGE NO.
1	INTRODUCTION	1
2	SYSTEM DESIGN	2-3
3	IMPLEMENTATION OVERVIEW	4-5
4	TESTING AND DEMONSTRATION	6-7
5	REFLECTION	8-9

INTRODUCTION

I chose a Library Management System. The older libraries are still manually managing everything with maintaining borrowing registers manually from many years. This can sometimes be tough for the librarian to manage so many records which sometimes lead to duplicacies, wrong informations about book availability or sometimes lost borrowing record.

To look out for the above mentioned problems I created a Library Managing system with python. My software would make it easy for the Librarian to manage the books, member, borrowing and returning of books. The system is well equipped with managing all the duplicacies, data and holding records smoothly.

The project was made with Object-Oriented Programming (OOP) principles with the use of 4 classes; Books, Members, Library and Loans. All the data was stored in the CSV files so that it was easier for the execution of the program to load and save the data from the operations. The project also has some additional features like automatic ID generations, Exception handling, duplicacy prevention and input validation for the improved and reliable usage of the system.

The focus and scope of the project is just the basic functionality for a small library management. The improvements can be done for a bigger Library with a better version of the the system.

SYSTEM DESIGN

The system is totally designed with OOP because it allows to keep the real entities like books, members and loans as different classes with their own attributes and methods.

The system consists of 4 main classes :

1. Book : The book class represents individual books stored in the library. It contains the attributes such as Book ID, Name, Author and Year of publication. It contains methods that allow books to be borrowed, returned, displayed and updated. The availability status makes sure that 1 book is not borrowed by multiple member at the same time.
2. Member : The member class deals with the members data stored in the Library. It has attributes like Member ID, Name, Email and borrowed books. The class have methods for borrowing books, returning books, displaying information and displaying currently borrowed books.
3. Loan : The loan class works with all the data about borrowing a book and returning it in the Library. It has Loan ID, Book ID, Member ID, Borrow date and Return date as it's attributes. And the methods in the Loan class are displaying loan info, getting the status of a loan, checking if the loan is active and marking a loan as returned.
4. Library : The library class is the home for all other classes. It acts as the controller of the system. This class provides the main functionality of application by adding books and members, searching records, borrowing books, returning books, displaying information, and generating unique IDs. Because of centralising all the operations the library class communicated with all other classes.

The project is divided into multiple python modules. Books.py, members.py, loans.py and library.py are the files that contain class definitions. Storage.py is the file responsible for handling data operations

such as loading data and saving data. Where, Main.py is the main menu of the system it contains the menu-driven user interface and manages everything with the user interaction.

Three different CSV files; Books.csv, members.csv and loans.csv are made for the data persistence. When the application is started, data gets loaded itself from the memory and when the application is closed it saves the data into the csv files automatically. This helps to preserve the data between program execution.

IMPLEMENTATION OVERVIEW

The main focus while implementation of the library system was on applying the fundamental programming concepts covered in the module. Several concepts like Object Oriented Programming, file handling, modular designing, exception handling, loops and conditional statements were used to make the system functional and maintainable.

Object Oriented Programming was the foundation of the system. Implementation of four classes was done with their respective attributes and methods that made it easier for the library environment and the code also became easier to organise and maintain. This approach also increased understanding and readability of the code by dividing different functions in different places.

Throughout the project methods were implemented to perform specific operations like borrowing-returning of books, display of the information, searching of records and generating the unique Ids. Loops were used for processing collection of books, loans and members. Loops were used for searching records, for displaying stored information and reconstructing of the active loans.

Conditional statements were also used to tackle the problem of duplicate records, to not let unavailable books to be borrowed and to detect invalid user input.

To improve reliability and user experience Exception handling was also implemented. Try-except blocks were used to avoid incorrect values in year, name and email and to avoid empty input. This helped to avoid the application from terminating and printing meaningful error messages for the user.

To improve maintainability, understandability, testing and extendibility in future the project was divided into multiple files. Each class was stored in its own file, while the functions for file handling were placed in a separate storage module.

Automated Generation of Book IDs and Member IDs were implemented for smooth functioning. Comments and Docstring were also added to increase the understanding of the code. PEP 8 style guidelines were followed for the improved readability and maintaining professional coding standards.

TESTING AND DEMONSTRATION

The application once completed was tested for the basic functionality and for the invalid user inputs being handled correctly.

```
===== LIBRARY MENU =====
1. Add Book
2. Add Member
3. Borrow Book
4. Return Book
5. Display Books
6. Display Members
7. Display Loans
8. Exit
Enter your choice: █
```

The first testig was for the adding books and members function whether they were being added successfully, automatic id generation was working or not and if duplicate entry was being added or not.

Then the borrowing and returning of books were tested. Testing for a loan being created when a member borrowed a book, looking for the return date behaviour while the book still being borrowed. Then if whether the book was marked reutrned and made avaialable for borrowing again.

```
===== LIBRARY MENU =====
1. Add Book
2. Add Member
3. Borrow Book
4. Return Book
5. Display Books
6. Display Members
7. Display Loans
8. Exit
Enter your choice: 2
Generated Member ID; M004
Enter Name: Divakar
Enter Email: Divakargmail.com
Invalid Name or Email.
```

Then the testing for invalid inputs was also done like for the name there should not be any number or symbols, the year of publicaion must only be numbers and if there was always an @ in the email. In each case appropriate error messages were displayed and valid input was requested.

Then the testing for empty Member or Book IDs while borrowing or returning books was also tested. For the books already borrowed the attempt to borrow them again was also tested but the error messages were successfully displayed.

```
===== LIBRARY MENU =====
1. Add Book
2. Add Member
3. Borrow Book
4. Return Book
5. Display Books
6. Display Members
7. Display Loans
8. Exit
Enter your choice: 3
Enter Member ID: M002
Enter Book ID: B002

--- Member Details ---
ID: M002
Name: Aston
Email: Aston@gmail.com

Borrowed Books:
No books borrowed.

--- Book Details ---
ID: B002
Title: Coading
Author: Aston
Year: 2005
Status: Available
Book borrowed successfully.
Loan L001 created successfully.
```

```
===== LIBRARY MENU =====
1. Add Book
2. Add Member
3. Borrow Book
4. Return Book
5. Display Books
6. Display Members
7. Display Loans
8. Exit
Enter your choice: 3
Enter Member ID: M002
Enter Book ID: B001

--- Member Details ---
ID: M002
Name: Aston
Email: Aston@gmail.com

Borrowed Books:
- B002

--- Book Details ---
ID: B001
Title: The Great Gatsby
Author: F. Scott Fitzgerald
Year: 1925
Status: Not Available
Book is not available for borrowing.
```


The data saving and loading was also tested by closing and restarting the application. All the data was being loaded to the respective CSV files and was also being restored between program executions.

```
loans.csv / data
1  Loan ID,Member ID,Book ID,Loan Date,Return Date
2  L001,M002,B002,2026-06-26,2026-06-26
3  L002,M001,B001,2026-06-26,
```

The testing process confirmed that the functional requirements were met by the system. All the features worked, appropriate error messages were displayed whenever needed making sure no space for errors while using the system.

```
===== LIBRARY MENU =====
1. Add Book
2. Add Member
3. Borrow Book
4. Return Book
5. Display Books
6. Display Members
7. Display Loans
8. Exit
Enter your choice: 8
Data saved successfully
Goodbye!
```

5. Reflection

The project was a great learning source for the practical knowledge and hands on experiencing. I learned a lot about how the programming concepts learned during the module were applied into a real world application.

With the project coming to end I had already gained so much practical knowledge about the Object-Oriented Programming. The division of multiple modules also showed the importance of a well written and organised code.

One of the main challenge that came forward while developing the application was the data reading files that were the CSV files. Initially I was having problems like records were not being loaded correctly due to some inconsistencies within the CSV files. Then I debugged it and got the understanding of file handling and data validation.

Another challenge was to keep maintaining the relationship between different classes when borrowing or returning the books. Testing and Debugging made it possible for the correct updation of the lists.

Moreover I learned the value of input validation and exception handling. The validation for year of publication, name, email, empty inputs and duplicate records also made the application user-friendly and more reliable. Instead of breaking the program an error message was displayed and the program asked for the correct input again.

Git and GitHub were completely new things for me. Its interesting to see how millions of coders can just put all their codes at one place and ask for help and help others with any problems. Using it I could now track any changes in my program throughout the development. It also provided a structured way to manage the project's source code and documentation. Preparing a README file in GitHub also made me realise that how important is the documenting of the software so that other users can also understand what my system is, why my system is, how does it run and everything else related to that.

A few developments that could be done if the project needs to be for a bigger library or developed further. A Graphical User Interface (GUI) can replace the command line interface to improve usability. A Relational Database such as MySQL can replace the CSV files to support bigger datasets and

better data management. Additional features such as Searching books by author or title, deleting or editing records, due-date reminders and overdue fine calculations can also be implemented.

To end, the project was really helpful in increasing my knowledge about Python programming, OOP, file handling, modular programming, debugging, testing and software designing. And also demonstrated the importance of documenting a software application